offer more confidence that the frontend changes can be deployed without integration problems. For example, we can use the Cypress framework to demonstrate how this could be engineered by substituting a Cypress fixture with a PactFlow stub, as shown in listing 9.1. Within the `before` statement of the test, a task is run to get the stub response data. The API request is intercepted, and the response is substituted with the stubbed response.

Listing 9.1 Cypress test with intercept

```
describe('Web consumer', () => {
    const endpoint = '/movies'
    before(() => {
        cy.task('getStub', endpoint).then((data) => {     #1
            cy.intercept(
            `http://localhost:4000${endpoint}`,     #2
              (req) => {
                req.reply(data)
            }).as('movies')
        })
    })

    it('List movies', () => {
        cy.visit(endpoint)
        cy.wait('@movies')
        cy.get('[data-cy="name"]').should('have.text', 'My movie')
    })
})
```

#1 getStub function shown in listing 9.2
#2 API service running on port 4000

The `getStub` task makes a request to Pact Broker with the API token header and returns the contract's expected response data as a stub.

Listing 9.2 Cypress plugin task

```
getStub(endpoint) {
      const result = axios.get(     #1
`${PACTFLOW_BROKER_URL}/pacts/provider/MoviesAPI/consumer/WebConsumer/version/lates
t/stub/${endpoint}`,                                  #2
        {
          headers: {
            Authorization: 'Bearer <PACTFLOW_API_TOKEN>',     #2
            Accept: 'application/json'
          }
        }).then(response => { return response.data })
      return result
    }
```

#1 Axios HTTP client
#2 PactFlow settings